

The Alberta Buck - Demurrage and the Jubilee Fund

Perry Kundert

2026-04-29

1 Overview

Alberta Buck imposes a flat 2%/yr demurrage on every BUCK in existence. The fee is **not** a transfer or a burn – it is a view-time computation against per-account state. The locked fees stay inside each account as inaccessible BUCKs, visible to the holder via `balanceOfFees(a)`. Aggregate demurrage is expressed via the Jubilee fund, an on-chain account whose raw ERC-20 balance grows **ONLY** at user-driven `mint()` / `burn()` events; between such events neither the Jubilee fund nor any user account experiences any state change due to demurrage.

The model has three load-bearing properties:

1. **Linear in time.** A BUCK held still earns demurrage at a linear rate on its raw balance: BUCK\$100 sitting unmoved for one year reports `balanceOf` = BUCK\$98. After 50 years it reports BUCK\$0. No exponential decay; no compounding.
2. **No demurrage burn.** No transfer or accrual event ever burns from a holder's `_balanceof.totalSupply` is mutated only by user-initiated `mint()` and `burn()` calls (and by Jubilee accrual that fires on those same calls).
3. **Conservation of effective spendable.** At any block, the sum of `balanceOf(a)` over every account (including Jubilee) plus the sum of `balanceOfFees(a)` equals `totalSupply` – no BUCKs are created or destroyed by demurrage; they are simply locked and slowly redistributed to the Jubilee fund at mint/burn checkpoints.

The system supports two transfer flavours, dispatched automatically by the sender's registered identity type:

- **Non-Carrying** (sender's `IdentityRegistry.isCarrying` flag is false – the default for EOAs registered via `register()`). The sender's accumulated BUCK-age is crystallized into its locked-fees pool; the recipient gets fresh BUCKs (no inherited fee debt).
- **Carrying** (sender's `isCarrying` flag is true – typical for service contracts bound via `bindContract()`). The sender does not crystallize on outflow; the carried portion's fee debt is added to the recipient's locked fees. Recipients accept Carrying transfers in advance via the standard identity-bound `approve(carryingAccount, 0, ...)` call – the same CP-receipt mechanism that gates any identity-bound transfer is also the consent to receive carried fees.

There is no separate `transferCarrying` function; both `transfer` and `transferFrom` dispatch through the registry flag. A contract has no will of its own – it executes on behalf of an operator and holds BUCK temporarily for others. The Carrying flavour pushes the demurrage on those

temporarily-held BUCKs to the parties that ultimately collect them, in exchange for whatever value the contract delivers.

Carrying-only contract accounts (e.g., the Jubilee fund, the Notes pool, AMM pools) do not pay demurrage from their own holdings – they deploy funds via ordinary **transfer** calls that the registry resolves to Carrying. Their goal is to keep only a "hot" working balance, so the demurrage on idle Carrying-account balance is a second-order effect.

The **isCarrying** flag is set at registration / binding time and is **immutable once any counterparty has approved the address**. The freeze prevents an operator from flipping a wallet's flavour after recipients have already consented to receive from it. Pre-approval reconfiguration (e.g., a freshly-deployed service contract changing its mind before it goes live) is permitted; once a single CP receipt names the address as spender, the flag is locked.

2 Per-account state

Every account **a** – EOAs, contract accounts, and the Jubilee fund – carries three storage values:

Field	Meaning
<code>_balanceof[a]</code>	Raw ERC-20 balance. Inherited from OZ ERC20. Mutated only by user-initiated balance changes (mint, burn, transfer in, transfer out). Demurrage does not modify it.
<code>_demurrage[a]</code>	Cumulative locked fees accrued up through <code>_timestamp[a]</code> .
<code>_timestamp[a]</code>	Block timestamp of the most recent balance-mutating event for a . All fees accrued through this instant are already folded into <code>_demurrage[a]</code> .

A freshly created account has `_balanceof = _demurrage = 0` and `_timestamp` unset (treated as zero, but no fee accrues against zero raw so the value is irrelevant until raw becomes non-zero).

3 Live fee calculation

Three view functions expose demurrage state. Let `BASE_RATE_PER_SEC = 2%/yr / (365.25 days)`, and let `now` be `block.timestamp`.

```
function feeOwing(a):
    elapsed = now - _timestamp[a]
    pending_fee = _balanceof[a] * BASE_RATE_PER_SEC * elapsed
    return _demurrage[a] + pending_fee

function balanceOfFees(a):
    return min(feeOwing(a), _balanceof[a])

function balanceOf(a):
    return _balanceof[a] - balanceOfFees(a)
```

Three behaviours fall out of these definitions:

1. `balanceOf` is what the holder can spend right now. It decreases linearly as time passes against a fixed raw balance. At BUCK\$100 raw and `BASE_RATE = 2%/yr`, after one day `balanceOf` = \$99.99452..., after one year = \$98, after fifty years = \$0.
2. `balanceOfFees` is the locked dust currently inaccessible to the holder. At any moment `balanceOf + balanceOfFees = _balanceof`.
3. `feeOwing` can in principle exceed `_balanceof` for an account that has been idle long enough; `balanceOfFees` caps the reported value at `_balanceof` (an account is never short more than it actually holds).

The view functions are $O(1)$ and require no on-chain state mutation – a wallet can compute live demurrage at any moment without spending gas.

4 Crystallization

Whenever an account's raw is about to change – mint into, burn from, transfer in, or transfer out – the contract calls a helper `_crystallize(a)` before the change:

```
function _crystallize(a):
    elapsed      = now - _timestamp[a]
    pending_fee  = _balanceof[a] * BASE_RATE_PER_SEC * elapsed
    _demurrage[a] = _demurrage[a] + pending_fee
    _timestamp[a] = now
```

The function is idempotent in time: calling it twice in the same block yields the same state as calling it once. After the call, `feeOwing(a) == _demurrage[a]` and the second term in the view is zero – the live fee is fully captured in storage.

Crystallization fires at the start of every event that touches an account's `_balanceof`. Pure read calls (`balanceOf`, `balanceOfFees`) never crystallize – they read storage and compute the view.

5 Transfers

The public surface is the standard ERC-20 `transfer(to, value)` and `transferFrom(from, to, value)`. `Buck._update` reads `IdentityRegistry.isCarrying(from)` and dispatches:

```
function _update(from, to, value):
    if from == 0 || to == 0:
        // mint / burn -- handled separately, see "Mint and burn"
        return _mintBurnUpdate(from, to, value)

    if identity.isCarrying(from):
        _carryingTransfer(from, to, value)
    else:
        _nonCarryingTransfer(from, to, value)
```

The flavour is determined entirely by the sender's registered identity type – it is never a per-call option. A user wanting to receive aged BUCK from a service contract approves that contract once via the existing identity-bound CP receipt; from then on every transfer that contract pushes to the user is Carrying.

5.1 Non-Carrying transfers (EOA sender)

When the sender is an EOA (or any address with `isCarrying == false`), the transfer crystallizes the sender's fees and lands fresh BUCKs at the recipient:

```
function _nonCarryingTransfer(from, to, value):
    require(value <= balanceOf(from))
    _crystallize(from)
    _crystallize(to)
    _balanceof[from] = _balanceof[from] - value
    _balanceof[to]   = _balanceof[to]   + value
    // _demurrage[from] retains the pre-existing locked fees.
    // _demurrage[to]   is unchanged: recipient gets fresh BUCKs.
    // Both _timestamp[*] are set to 'now' by crystallization.
```

Operational invariants:

- The sender's residual BUCKs continue to age from `now` forward (the fresh `_timestamp[from]` starts a new linear accrual against the smaller raw).
- The sender's accumulated locked fees stay locked in the sender's `_demurrage[from]`. They are NOT moved to the recipient and they are NOT moved to the Jubilee fund.
- The recipient's incoming BUCKs land at age 0 – the recipient's `_demurrage[to]` does not change.

The check `value <= balanceOf(from)` prevents the sender from spending its locked fees: only `balanceOf(from)` is available for outflow. The OZ super `_update` guards on raw, so the extra spendable check sits in front.

5.2 Carrying transfers (Carrying-flagged sender)

When the sender's `isCarrying` flag is true (typically a service contract: the Jubilee fund, a Notes pool, an AMM pool), the same `transfer` / `transferFrom` dispatches through the internal Carrying path. The transferred portion's demurrage is pushed onto the recipient's locked fees; the sender's basis is left untouched.

```
function _carryingTransfer(from, to, value):
  require(value <= _balanceOf[from])
  age_basis_seconds = now - _timestamp[from]
  carried_fee      = value * BASE_RATE_PER_SEC * age_basis_seconds
  _crystallize(to)
  _balanceOf[from] = _balanceOf[from] - value
  _balanceOf[to]   = _balanceOf[to]   + value
  _demurrage[to]   = _demurrage[to]   + carried_fee
  // _timestamp[from] and _demurrage[from] are unchanged --
  // the residual continues to age from the original basis.
```

Operational invariants:

- The sender's `_timestamp[from]` and `_demurrage[from]` do not change. The residual BUCK in `_balanceOf[from]` continues to accrue fee at the same age basis – no fresh clock for the remaining balance.
- The recipient's `_demurrage[to]` absorbs the carried fee: `balanceOf(to)` immediately reflects the inherited debt. The recipient's `_balanceOf[to]` grows by the gross value.
- The Carrying transfer is its own balance-mutating event for `to`, so the recipient is crystallized first; the carried fee is then added to the just-refreshed `_demurrage[to]`.

A Carrying transfer is only valid when the recipient has explicitly consented (see "Approve and consent" below). This prevents a malicious Carrying account from dumping unwanted fee debt on an unconsenting counterparty.

5.3 Algebraic identity: total spendable is preserved

Both transfer flavours preserve $\text{sum_a } \text{balanceOf}(a) + \text{sum_a } \text{balanceOfFees}(a) = \text{totalSupply}$ block-by-block. The distribution of locked fees across accounts differs:

- Non-Carrying: the sender's locked grows; the recipient lands fresh.
- Carrying: the recipient's locked grows by the carried amount; the sender's locked stays at whatever it was.

Both keep `totalSupply` unchanged and conserve the system fee debt.

6 Mint and burn

Only `mint()` and `burn()` can mutate `totalSupply`. Both trigger Jubilee accrual before performing the user’s intent.

6.1 Sequence on `Buck.mint(amount)`

```
function mint(amount):
  require(identity.isVerified(msg.sender))
  require(amount + _balanceof[msg.sender] <= storedLimit[msg.sender])
  _accrueJubilee() // see below
  _crystallize(msg.sender)
  premium = _computePremium(msg.sender, amount, storedLimit[msg.sender])
  super._mint(msg.sender, amount - premium)
  if premium > 0:
    _crystallize(insurancePool)
    super._mint(insurancePool, premium)
```

6.2 Sequence on `Buck.burn(amount)`

```
function burn(amount):
  require(amount <= balanceOf(msg.sender))
  _accrueJubilee()
  _crystallize(msg.sender)
  super._burn(msg.sender, amount)
```

6.3 Jubilee accrual

The Jubilee fund’s raw balance grows **ONLY** when `_accrueJubilee()` runs, and `_accrueJubilee()` runs **ONLY** at the start of `mint()` or `burn()`.

```
uint256 internal _jubileeLastUpdate; // initialised to genesis

function _accrueJubilee():
  elapsed = now - _jubileeLastUpdate
  if elapsed == 0:
    return
  delta = totalSupply() * BASE_RATE_PER_SEC * elapsed
  _crystallize(address(this))
  super._mint(address(this), delta)
  _jubileeLastUpdate = now
```

The accrual mints fresh BUCK to `address(this)` equal to the 2%/yr rate applied to the **current** total supply across the elapsed interval (the supply is piecewise constant between mint/burn events, so the integral is just `totalSupply * elapsed`). `totalSupply` grows by `delta`; the new BUCK is backed by the locked fees that already exist in user accounts.

After the accrual:

- `_balanceof[address(this)]` is larger.
- `_timestamp[address(this)]` is **now** (the crystallization step folds Jubilee’s prior age into its `_demurrage[]` before the new BUCKs land).
- The new BUCKs themselves are at age 0; they will accrue Jubilee’s own self-demurrage going forward.

6.4 Why the Jubilee fund cannot be a pure view

Earlier sketches considered making `balanceOf(jubilee)` a computed view (`BASE_RATE * area_under_supply` minus a self-demurrage haircut) with no actual ERC-20 storage. That fails when the Jubilee must transfer out: a BuckCredit lien redemption needs to **burn** BUCKs from the Jubilee account, and you cannot burn from a virtual balance. The Jubilee therefore holds real ERC-20 storage that grows at mint/burn checkpoints.

7 The Jubilee fund

The Jubilee fund lives at `address(this)` (the Buck contract itself). It is a Carrying account: it does not pay its own demurrage from its own holdings on outflow; instead it pushes funds via Carrying transfers, and recipients absorb the proportional fee debt.

Holdings and operations:

- `_balanceof[address(this)]` grows at every mint/burn via `_accrueJubilee()`.
- `balanceOf(address(this)) = _balanceof[address(this)] - feeOwing(address(this))`. Just like any other account, the Jubilee accrues its own demurrage on its idle balance.
- `balanceOfFees(address(this))` is the Jubilee's locked self-demurrage. These BUCKs stay in the Jubilee until carried out via a future transfer to a consenting recipient.
- The Jubilee deploys funds via the ordinary `Buck.transfer(to, amount)` called by an authorised governance process; because `isCarrying[address(this)]` is true at binding time, `Buck._update` dispatches through the Carrying path automatically. Recipients must have consented via the identity-bound `approve(jubilee, 0, ...)`.

Practical posture: the Jubilee should be lean. Funds it collects are intended to be deployed promptly to the insurance pool, BUCK_K stabilization mechanisms, public goods, etc., so that the "hot" balance held idle is small and self-demurrage on it is negligible. Long idleness produces a real but bounded degradation in `balanceOf(jubilee)` that is recovered via Carrying transfer to consenting recipients.

8 Worked example: one BUCK\$100 mint, no other events

Setup at $t=0$:

```
mint(100) by alice
state:
  alice:    _balanceof=100, _demurrage=0,    _timestamp=0
  jubilee:  _balanceof=0,   _demurrage=0,    _timestamp=0
totalSupply: 100
```

After exactly 1 year (no other events):

```
state (storage unchanged):
  alice:    _balanceof=100, _demurrage=0,    _timestamp=0
  jubilee:  _balanceof=0,   _demurrage=0,    _timestamp=0
totalSupply: 100
```

```
views:
  feeOwing(alice)      = 100 * 2%/yr * 1yr = $2
```

```

balanceOf(alice)      = $100 - $2 = $98
balanceOfFees(alice)  = $2

feeOwing(jubilee)     = 0 * 2%/yr * 1yr = $0
balanceOf(jubilee)    = $0

```

The "missing" BUCK\$2 is locked in alice's account and will be effectively "swept" into the Jubilee fund at the next mint/burn event (the Jubilee fund will grow by the same value locked in alice's account).

If at exactly $t=1\text{yr}$ a second account does `mint(1)`:

```

1) _accrueJubilee():
   elapsed = 1yr, totalSupply = $100
   delta   = $100 * 2%/yr * 1yr = $2
   _crystallize(jubilee): _demurrage += 0
   super._mint(jubilee, $2): _balanceof += $2, totalSupply = $102
   _timestamp[jubilee] = t1
2) _crystallize(alice): _demurrage += $2, _timestamp = t1
3) super._mint(bob, $1): bob._balanceof = $1, totalSupply = $103

```

```

state after:
alice: _balanceof=$100, _demurrage=$2, _timestamp=t1
bob:   _balanceof=$1,   _demurrage=0,   _timestamp=t1
jubilee: _balanceof=$2, _demurrage=0,   _timestamp=t1
totalSupply: $103

```

```

views (instant after the mint):
balanceOf(alice) = 100 - 2 = $98
balanceOf(bob)   = $1
balanceOf(jubilee) = $2

sum balanceOf = $98 + $1 + $2 = $101
sum balanceOfFees = $2 (alice) + 0 + 0 = $2
total           = $103 = totalSupply

```

After a further year (no events between t_1 and $t_1+1\text{yr}$):

```

state (storage unchanged):
alice: _balanceof=$100, _demurrage=$2, _timestamp=t1
bob:   _balanceof=$1,   _demurrage=0,   _timestamp=t1
jubilee: _balanceof=$2, _demurrage=0,   _timestamp=t1

views at t1+1yr:
feeOwing(alice) = 2 + 100 * 2%/yr * 1yr = $4
balanceOf(alice) = $96

feeOwing(bob)   = 0 + 1 * 2%/yr * 1yr = $0.02
balanceOf(bob)  = $0.98

feeOwing(jubilee) = 0 + 2 * 2%/yr * 1yr = $0.04
balanceOf(jubilee) = $1.96

sum balanceOf = $96 + $0.98 + $1.96 = $98.94
sum balanceOfFees = $4 + $0.02 + $0.04 = $4.06
total          = $103 = totalSupply

```

The Jubilee held \$2 raw at t_1 , and at $t_1+1\text{yr}$ reports `balanceOf` = \$1.96 – \$2 raw minus its own \$0.04 of self- demurrage on twelve months at age 1yr. The earlier illustrative figure of \$1.98 corresponds to a smoother continuous-accrual approximation in which the Jubilee's age is six-month average; in the actual implementation the average age depends on the spacing of mint/burn events.

9 Carrying accounts and the `isCarrying` flag

A Carrying account is any address whose `IdentityRegistry.isCarrying[a]` flag is true. The account itself accrues demurrage like any other; the distinguishing behaviour is on outflow, where `Buck._update` dispatches the sender's transfer through the Carrying path automatically.

9.1 Setting the flag

The flag lives in `IdentityRegistry` and is set at registration time:

Path	Default	Override
<code>register()</code>	false (EOA)	not permitted – EOAs are always Non-Carrying.
<code>bindContract()</code>	true (service contract)	<code>bindContract(target, pk, E, isPublic, /isCarrying=false/)</code> for contract wallets that should crystallize like an EOA; multisigs operated by a single user, AA wallets, etc..

Carrying contracts are expected to be rare in practice – most contract bindings are service contracts (Notes pools, AMM pools, the Jubilee fund itself) where the flag defaults true. Smart-contract wallets opt out at binding time.

9.2 Immutability after first approval

Once any counterparty issues an identity-bound approve naming this address as the spender (i.e., a CP receipt fragment is written for the (counterparty, a) pair), the `isCarrying[a]` flag is locked. Pre-approval changes are permitted (a freshly-deployed contract may flip its mind before going live); post-approval changes are not (recipients have already consented to a specific Carrying flavour and the operator must not be able to retroactively change it).

The lock is enforced by a per-address counter or boolean `_carryingFrozen[a]` maintained by `IdentityRegistry`:

```
function approve(spender, amount, E_spender, pi_CP):  
    // ... existing CP-receipt machinery ...  
    identity._carryingFrozen[spender] = true  
  
function setIsCarrying(target, value):  
    require(msg.sender == bindingOperator(target))  
    require(!identity._carryingFrozen[target], "isCarrying frozen by approval")  
    identity.isCarrying[target] = value
```

A re-bound (re-deployed) contract resets through the standard `bindContract()` flow; in-place flag changes after first approval are simply not supported.

9.3 Why Carrying for service contracts

A service contract holds BUCK temporarily on behalf of others (the Jubilee fund pre-deployment; a Notes pool holding face value until a note is spent; an AMM liquidity pool waiting to be paired). If such an account were forced to crystallize on outflow like an ordinary EOA, it would have to surface its own accumulated demurrage and either absorb it (degrading the funds it manages) or push it onto the recipient as a non-Carrying gift of fresh BUCKs (mis-pricing the actual age-adjusted value). The Carrying flavour preserves the age basis: BUCK that has been sitting in the Notes pool for six months arrives at the redeemer with six months of age-adjusted fee debt baked in, and the recipient sees a `balanceOf` less than the gross face accordingly.

9.4 Sender side of a Carrying transfer

The sender's `_timestamp` and `_demurrage` are unchanged. Sender's `_balanceof` drops by `value`. The residual continues to age from the same basis – it does NOT start a fresh clock.

9.5 Recipient side of a Carrying transfer

Recipient is crystallized (any pending fees on its prior `_balanceof` are folded into `_demurrage[to]`, `_timestamp` is reset to now). Then the carried fee value * (now - sender_timestamp) * BASE_RATE is added to `_demurrage[to]`, and value is added to `_balanceof[to]`.

The instantaneous post-transfer effect:

```
balanceOf(to)    = _balanceof[to] (post) - _demurrage[to] (post)
                  = (raw_old + value) - (existing_locked + carried_fee)
                  = balanceOf_old + (value - carried_fee)
```

The recipient's spendable goes up by the **net** amount `value - carried_fee`, exactly the age-adjusted value of the incoming BUCKs.

10 Approve and consent

The identity-bound `Buck.approve(spender, amount, E_spender, pi_CP)` serves a dual role under this model. Its primary job is unchanged – it produces the CP receipt fragment that gates identity-bound transfers between the (sender, spender) pair. Its second job is to express the counterparty's consent to receive Carrying transfers from `spender` if `spender` is (or becomes) Carrying-flagged.

10.1 One approve covers both directions

The CP receipt is checked symmetrically for any transfer between the two identities, regardless of which one ends up as `msg.sender`. When the spender is a service contract (`isCarrying = true`) and pushes BUCK to the recipient, the recipient's prior approve has already established the receipt; the contract's Carrying transfer proceeds through the same identity-bilateral check as any other identity-bound transfer.

10.2 Zero-allowance approves for receive-only consent

When a recipient simply wants to enable an incoming Carrying transfer (e.g., to receive future deployments from the Jubilee fund) without granting any spend allowance, they call:

```
recipient calls:
  Buck.approve(carryingAccount, 0, E_carryingAccount, pi_CP)
```

This creates the receipt fragment without granting allowance. `amount=0` is exactly the semantic the user wants: identity binding plus consent-to-receive without spending authorisation.

10.3 Approval freezes isCarrying

Per the immutability rule above, the first approve naming `carryingAccount` as the spender freezes `isCarrying[carryingAccount]`. A wallet UI presenting an approve confirmation should surface the current `isCarrying` value so the user can see whether they're consenting to a Carrying-flavour relationship; once they approve, the operator can no longer change the flavour.

10.4 Non-Carrying senders

The existing identity-bound approve applies unchanged: a non-Carrying sender approves a spender with a non-zero allowance and a CP receipt; `transferFrom` checks both the allowance and the receipt. Non-Carrying transfers carry no fee debt to the recipient – the recipient receives fresh BUCKs – so no extra consent dimension is involved.

11 Lien redemption

A BuckCredit NFT backs a BUCK mint via an off-chain insured asset. When the BuckCredit's "close" mechanic is invoked (asset returned, lien released), it must reverse the original mint by burning BUCK equal to the originally-insured face value. The burn is sourced jointly from the original holder (alice) and the Jubilee fund, in proportion to the elapsed fraction of the credit's lifetime.

11.1 Mechanics

For a BuckCredit insured at face value F with lifetime L , closed at elapsed time e (with $e \leq L$):

```
function close(BuckCreditId):
  F = creditFace(id)
  L = creditLifetime(id)
  e = now - creditMintTime(id)

  fromJubilee = F * min(e, L) / L
  fromAlice   = F - fromJubilee

  Buck._accrueJubilee()           // bring Jubilee current
  Buck._crystallize(alice)
  Buck._crystallize(jubilee)

  require(_balanceof[alice] >= fromAlice, "alice underfunded")
  require(_balanceof[jubilee] >= fromJubilee, "jubilee underfunded")

  Buck.super._burn(alice, fromAlice)
  Buck.super._burn(jubilee, fromJubilee)
  // BuckCredit NFT then releases the asset to alice and self-burns.
```

11.2 Worked example: 50-year credit closed at year 50

Alice mints BUCK\$100 at $t=0$ (let $F=100$, $L=50\text{yr}$, no premium for simplicity). No further events for 50 years.

At $t=50\text{yr}$, BuckCredit calls `close`:

```
1) _accrueJubilee():
  elapsed = 50yr
  delta   = 100 * 2%/yr * 50yr = $100
  _crystallize(jubilee): no-op (raw=0 since genesis)
  super._mint(jubilee, $100): _balanceof[jubilee] = $100
  totalSupply: $100 -> $200

2) _crystallize(alice):
  pending_fee = 100 * 2%/yr * 50yr = $100
  _demurrage[alice] = $100, _timestamp[alice] = t50

3) Compute split:
  fromJubilee = $100 * 50/50 = $100
  fromAlice   = $100 - $100 = $0
```

- 4) Burns:


```
super._burn(alice, $0): no change
super._burn(jubilee, $100): _balanceof[jubilee] = 0
totalSupply: $200 -> $100
```
- 5) BuckCredit releases the asset to alice and self-destructs.

Alice's `_balanceof` remains \$100, but `_demurrage[alice] = $100` so `balanceOf(alice) = $0`. The original mint has been fully redeemed: alice paid the full \$100 of demurrage over 50 years (it accrued in her `_demurrage[alice]` at lien-close time), and the Jubilee fund burned the matching \$100 it had accumulated.

11.3 Worked example: closed at year 25 (half-elapsed)

Same setup, close at `t=25yr`:

- 1) `_accrueJubilee()`:


```
delta = 100 * 2%/yr * 25yr = $50
super._mint(jubilee, $50): _balanceof[jubilee] = $50
totalSupply: $100 -> $150
```
- 2) `_crystallize(alice)`:


```
pending_fee = 100 * 2%/yr * 25yr = $50
_demurrage[alice] = $50, _timestamp[alice] = t25
```
- 3) Compute split:


```
fromJubilee = $100 * 25/50 = $50
fromAlice   = $100 - $50   = $50
```
- 4) Burns:


```
super._burn(alice, $50): _balanceof[alice] = $50
super._burn(jubilee, $50): _balanceof[jubilee] = 0
totalSupply: $150 -> $50
```
- 5) Asset returned, BuckCredit destroyed.

After early closure alice retains \$50 of raw BUCK with `_demurrage[alice]` still \$50 – giving `balanceOf(alice) = $0` immediately. The implementation may choose to clear `_demurrage[alice]` as part of `close`, in which case alice walks away with \$50 spendable; this is a policy decision tied to BuckCredit's specific close terms. (Discuss further with the BuckCredit design.)

11.4 Why no compound interest is required

The integral identity that makes the redemption work cleanly: `BASE_RATE * totalSupply * elapsed` is exactly the amount Jubilee can offer at lien close, and it equals exactly what the holder has accrued in locked fees over that elapsed period. Both sides sum to `F` over `L` for a credit that ages its full lifetime. There is no need for any account to compound interest on demurrage; the rate is applied to raw `_balanceof` linearly, and Jubilee's accumulation tracks the same linear integral across all outstanding BUCK.

12 Conservation invariants

Verifiable on every block, after every operation:

1. Raw conservation.

`sum_a _balanceof[a] = totalSupply`

ERC-20 super-state. `mint()`, `burn()`, and `_accrueJubilee()` are the only writes that move `totalSupply`.

2. **Effective spendable identity.**

`sum_a balanceOf(a) + sum_a balanceOfFees(a) = totalSupply`

Direct from `balanceOf = _balanceof - balanceOfFees` and `balanceOfFees <= _balanceof`.

3. **Demurrage rate identity.** Between any two adjacent mint/burn events at times `t_0` and `t_1` (`totalSupply` constant):

`d(sum_a balanceOfFees(a)) / dt = BASE_RATE * totalSupply`

The locked-fee total grows at exactly the demurrage rate times the supply. Across multiple intervals this integrates to `BASE_RATE * area_under_supply`, which is the cumulative Jubilee target. When `_accrueJubilee()` materializes a delta, it shifts that growth from "locked in user accounts" to "live in `_balanceof[address(this)]`" without changing any sum.

4. **Carrying-transfer invariant.** A transfer whose sender has `isCarrying = true` leaves both sides unchanged in `sum balanceOf` and `sum balanceOfFees` (apart from rounding). The locked-fee total is conserved, with the carried amount moving from sender's implicit basis to recipient's explicit `_demurrage[to]`.
5. **Non-Carrying transfer invariant.** A transfer whose sender has `isCarrying = false` leaves `sum balanceOf` unchanged; `balanceOfFees(from)` crystallizes into `_demurrage[from]` (no change in the reported view). Sum totals are preserved.
6. **isCarrying-immutability invariant.** For any address `a` such that `_carryingFrozen[a]` is true, `isCarrying[a]` cannot be modified by any subsequent transaction. `_carryingFrozen[a]` becomes true the first time `a` appears as the spender argument of an `approve()` call.

13 Comparison with a "pure-view Jubilee" model

An earlier draft considered a model in which the Jubilee fund had no ERC-20 storage at all; `balanceOf(address(this))` would be a computed view (`BASE_RATE * area_under_supply` minus a self-demurrage haircut), and no `_accrueJubilee` would mint fresh BUCK. Reasons that model was rejected:

- **Lien redemption.** `BuckCredit.close` must burn BUCKs from the Jubilee. A virtual balance cannot be burned without inventing a parallel supply concept; making the Jubilee a real ERC-20 holder is operationally simpler and matches user intuition.
- **Carrying transfers from Jubilee.** A Carrying transfer pushes `_balanceof[jubilee]` down by `value` and the recipient's `_balanceof[to]` up. Both are real ERC-20 movements; both sides must be backed by real raw.

- **Inflation matches lockedness.** The fresh BUCK minted to Jubilee at every checkpoint exactly equals the amount of locked fees that have accumulated in user accounts over the same interval. `totalSupply` grows; user `balanceOf`s shrink; the gap appears as Jubilee's raw balance. The bookkeeping is local and inspectable.

The current model accepts `totalSupply` growth between `mint`/`burn` events as the cost of having a real Jubilee account that can transact. The growth is bounded by the locked-fee identity above and is fully unwound at `lien close`.

14 Implementation sketch

14.1 Buck.sol storage

```
mapping(address => uint256) internal _demurrage;
mapping(address => uint64)   internal _timestamp;
uint64                      internal _jubileeLastUpdate;
```

Per-account `_balanceOf[a]` continues to be inherited from OZ ERC20. The previous `_indexAtLastTouch[a]` mapping (cumulative- index style) is replaced by `(_demurrage[a], _timestamp[a])` per the model above. The previous `_areaAcc`, `_areaLastUpdate` and `_cumIndex` storage is no longer needed: the Jubilee accrual reads `totalSupply()` and the elapsed time directly.

14.2 IdentityRegistry storage additions

```
mapping(address => bool) public  isCarrying;
mapping(address => bool) internal _carryingFrozen;
```

`isCarrying[a]` is set at registration / binding time and read on every `Buck._update` to dispatch the transfer flavour. `_carryingFrozen[a]` is set to true the first time `a` appears as the spender on an `approve()` call; once frozen, `isCarrying[a]` cannot be changed.

14.3 Buck.sol public/external view surface

```
function feeOwing(address a)           external view returns (uint256);
function balanceOfFees(address a)      external view returns (uint256);
function balanceOf(address a)          public view override returns (uint256);
function rawBalanceOf(address a)       external view returns (uint256);
function jubileeBalance()              external view returns (uint256);
```

(`jubileeBalance()` is just an alias for `balanceOf(address (this))` – there is no separate "target" view because target no longer exists as a distinct concept; it is implicit in the locked-fee sum.)

There is **no** `transferCarrying` public function. The two transfer flavours are dispatched internally based on `IdentityRegistry.isCarrying(msg.sender)` (or from for `transferFrom`).

14.4 Buck.sol internal helpers

```
function _crystallize(address a) internal;
function _accrueJubilee()         internal;
function _nonCarryingTransfer(address from, address to, uint256 value) internal;
function _carryingTransfer(address from, address to, uint256 value)   internal;
```

14.5 ERC-20 hook dispatch

```
function _update(address from, address to, uint256 value) internal override {
    if (from == address(0) || to == address(0)) {
        // mint / burn -- _accrueJubilee then super._update; see "Mint and burn".
        _accrueJubilee();
        if (from != address(0)) _crystallize(from);
        if (to != address(0)) _crystallize(to);
        super._update(from, to, value);
        return;
    }

    if (identity.isCarrying(from)) {
        _carryingTransfer(from, to, value);
    } else {
        _nonCarryingTransfer(from, to, value);
    }
}
```

14.6 IdentityRegistry API additions

```
// register() always sets isCarrying[msg.sender] = false.
// bindContract() takes a default-true isCarrying flag, with an
// optional override for non-service contracts (multisigs, AA wallets).
function bindContract(
    address target,
    BN254.G1Point calldata pk,
    ElGamalCT calldata E,
    bool isPublicIdentity_,
    bool isCarrying_
) external;

// Explicit setter for pre-approval reconfiguration. Reverts if
// _carryingFrozen[target] is already set.
function setIsCarrying(address target, bool value) external;

// Called from Buck.approve() to freeze the spender's flag.
function _markApproved(address spender) internal;
```

Receipt fragments and identity binding (existing `IdentityRegistry` flow) are layered on top unchanged: the identity check for both flavours of transfer happens at the public function boundary, before the demurrage logic runs. `Buck.approve` now also calls `identity._markApproved(spender)` so the spender's `isCarrying` flag becomes immutable from that moment on.

15 Open questions and policy choices

These are decisions to be settled in companion documents or in the implementation:

15.1 Should close clear `_demurrage` on the redeeming account?

After early closure ($e < L$) the holder's `_demurrage[a]` still reflects the locked fees from before the close. The holder retains some raw, but `balanceOf(a)` reads as zero until those fees re-amortize. The implementation may choose to zero `_demurrage[a]` as part of `close`, freeing the residual immediately, or to leave it in place and let it re-amortize over the remaining "credit-free" period. This is a BuckCredit policy choice, not a Buck demurrage primitive.

15.2 What happens if a Carrying account is unfunded at close time?

If the Jubilee fund's `_balanceof` is insufficient to satisfy `fromJubilee` (because someone else minted between the `_accrueJubilee()` step and the burn – in practice impossible within a single transaction, but worth specifying), the close should revert and the `BuckCredit` should be re-tried in a later block. The mint/burn-only Jubilee accrual is sufficient to keep the fund solvent at the moment of any close.

15.3 Multiple BuckCredits and shared Jubilee

The Jubilee fund is a single pool serving all `BuckCredits` simultaneously. Its accrual rate is on `totalSupply`, not on any per-credit notion. A close consumes Jubilee funds in proportion to the elapsed fraction of the closing credit's lifetime; the fund is replenished at every subsequent `mint/=burn` elsewhere in the system. The model degrades gracefully under heavy redemption load: if many credits close at once, `_accrueJubilee()` simply tracks the cumulative `totalSupply * elapsed` integral and Jubilee's `_balanceof` covers the joint demand.

15.4 Long-idle Carrying accounts

A Carrying account whose policy is to hold BUCK indefinitely will accumulate self-demurrage that depresses its `balanceOf` relative to its raw. This is intended behaviour – it incentivises Carrying accounts to deploy funds rather than hoard. However, very long idle periods can produce `balanceOf(carrying) << _balanceof(carrying)`, and a Carrying transfer of the full raw would push a large carried fee onto the recipient. The contract enforces only the technical constraint `value <= _balanceof`; recipient consent (via the zero-allowance approve) is the social control.

15.5 Premium routing and the insurance pool

The mint `premium` computed by `_computePremium` is routed to `insurancePool`, a non-Carrying account. The insurance pool accrues demurrage on its holdings like any other account; when it deploys funds it does so via standard non-Carrying `transfer`. No special handling is required.

16 References

- `src/Buck.sol` – the implementation following this design.
- `src/IdentityRegistry.sol` – identity binding and approve receipts.
- `alberta-buck-ethereum.org` – the umbrella design document introducing BUCK_K and the `BuckCredit` insurance machinery.
- `alberta-buck-identity-example.org` – worked cryptographic examples for the identity-bound approve flow.